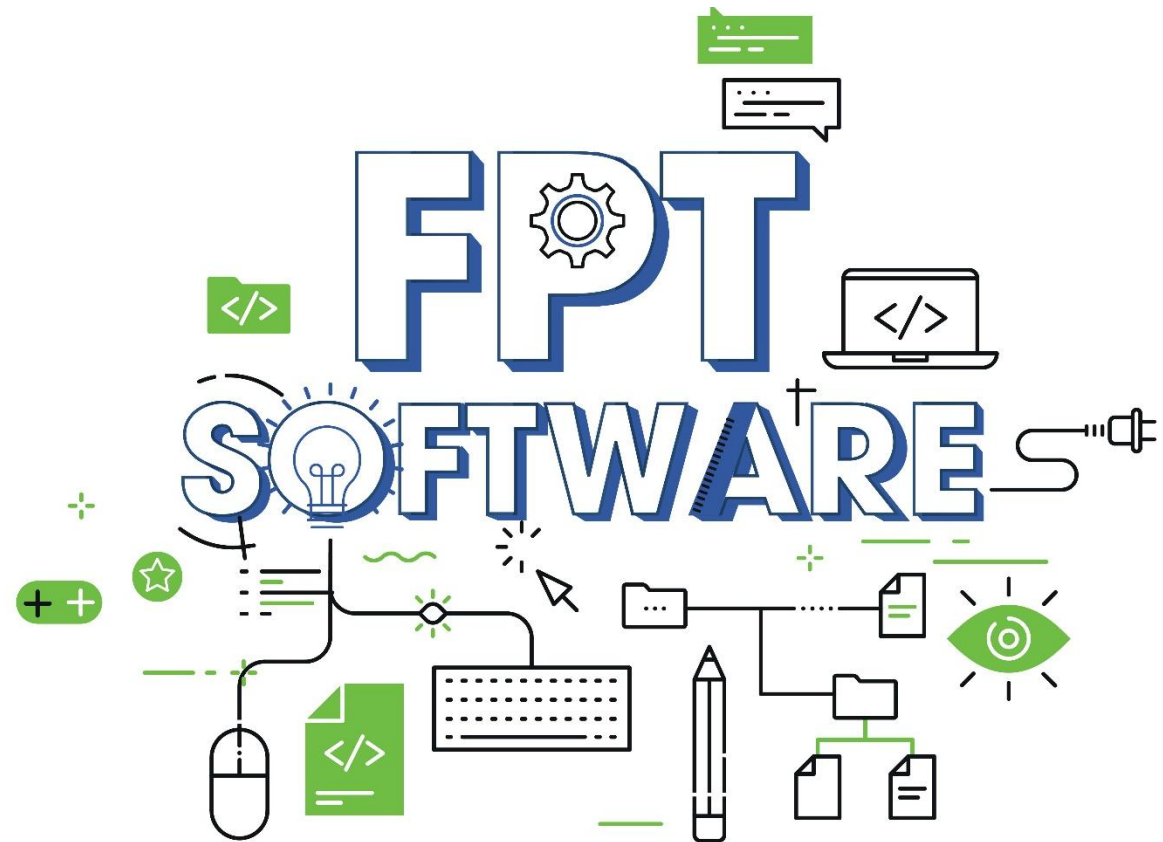


Basic Android

Intent & Broadcast Receiver



Agenda

1. Intent

2. Broadcast Receiver





Intent

Learning Goals

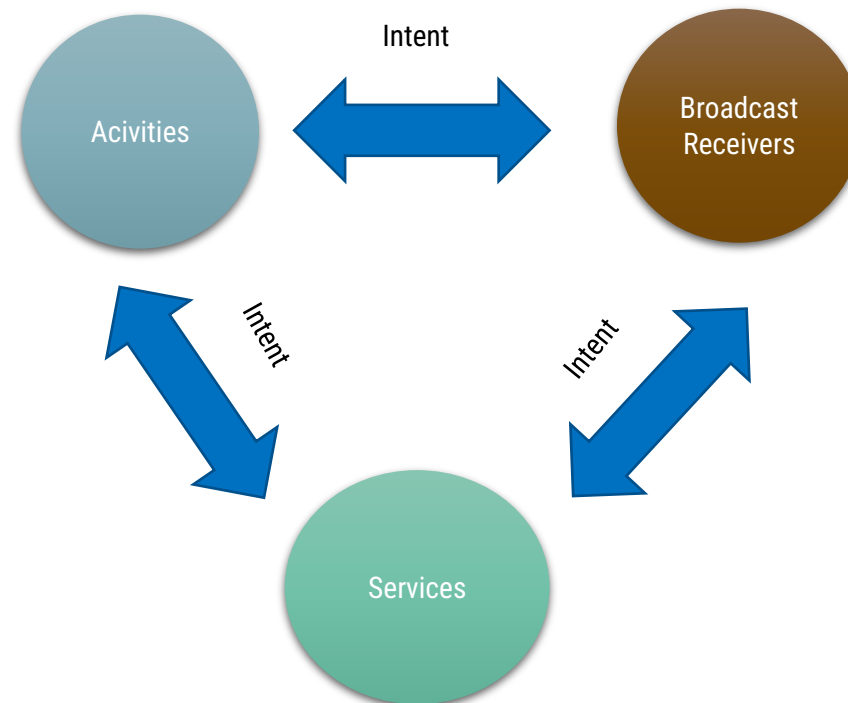
After the course, attendees will be able to:

- ▶ Understanding about Intent and implement of Intent in Android
- ▶ Understanding about Broadcast Receiver and implement of Broadcast Receiver in Android

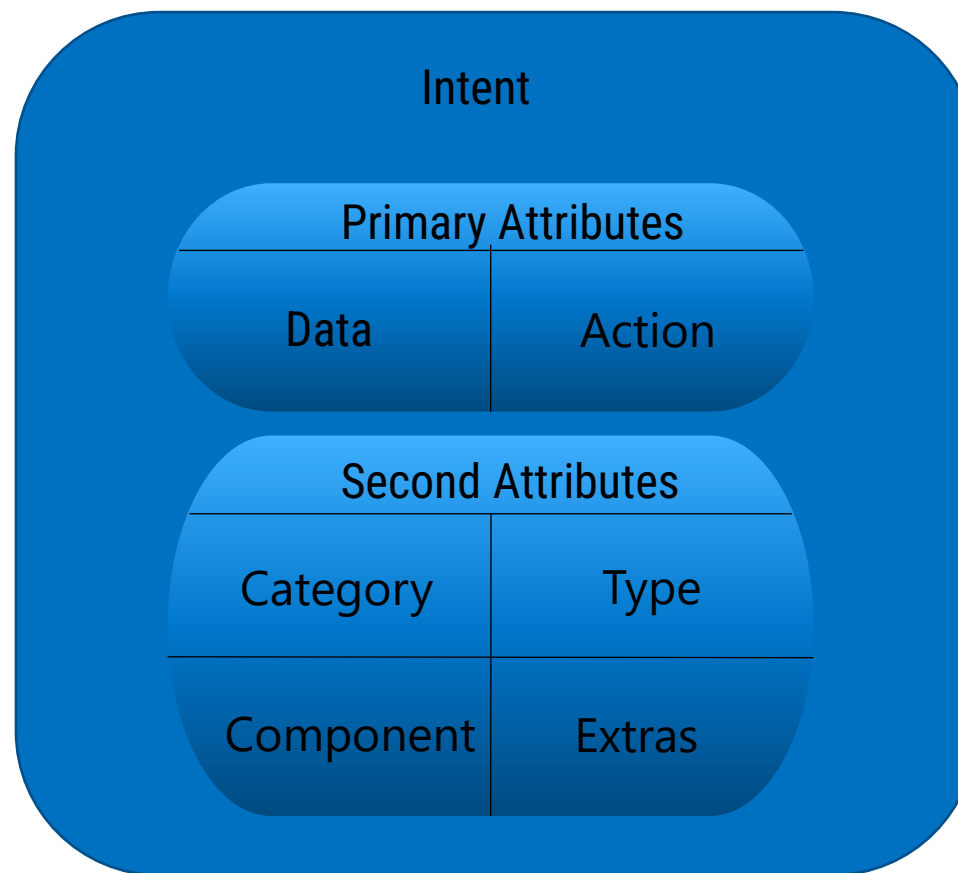
What is intent?

Intent is an abstract description of an operation to be performed.

- ▶ Use to Start an activity
- ▶ Use to send to any BroadCast Receiver
- ▶ Use to communicate with a service



Intent Structure



▶ Action Field

- A string naming to describe the action to be performed. Such as [ACTION_VIEW](#), [ACTION_EDIT](#), [ACTION_MAIN](#), etc.

```
public static final String ACTION_MAIN = "android.intent.action.MAIN";  
public static final String ACTION_VIEW = "android.intent.action.VIEW";  
....
```

- We can also define your own action strings for activating the components in your

```
public static final String ACTION_FINISH_DISPLAY = "com.ivi.mirror.service.FINISH_DISPLAY";  
....
```

▶ Data

- The data to operate on, such as a person record in the contacts database, expressed as a [Uri](#).

```
Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse(uriString: "https://www.google.com/"));  
startActivity(intent);
```

► Category

- Gives additional information about the action to execute. For example, CATEGORY_LAUNCHER means it should appear in the Launcher as a top-level application, while CATEGORY_ALTERNATIVE means it should be included in a list of alternative actions the user can perform on a piece of data
- Android provides a set of predefined categories, besides we can define your own categories

```
Intended to be used as a tab inside of a containing TabActivity.  
  
1 usage  
@SdkConstant(SdkConstantType.INTENT_CATEGORY)  
public static final String CATEGORY_TAB = "android.intent.category.TAB";  
  
Should be displayed in the top-level launcher.  
  
@SdkConstant(SdkConstantType.INTENT_CATEGORY)  
public static final String CATEGORY_LAUNCHER = "android.intent.category.LAUNCHER";  
  
Indicates an activity optimized for Leanback mode, and that should be displayed in the Leanback launcher.  
  
@SdkConstant(SdkConstantType.INTENT_CATEGORY)  
public static final String CATEGORY_LEANBACK_LAUNCHER = "android.intent.category.LEANBACK_LAUNCHER";
```

► Type

- Specifies an explicit type (a MIME type) of the intent data. Normally the type is inferred from the data itself. By setting this attribute, you disable that evaluation and force an explicit type.

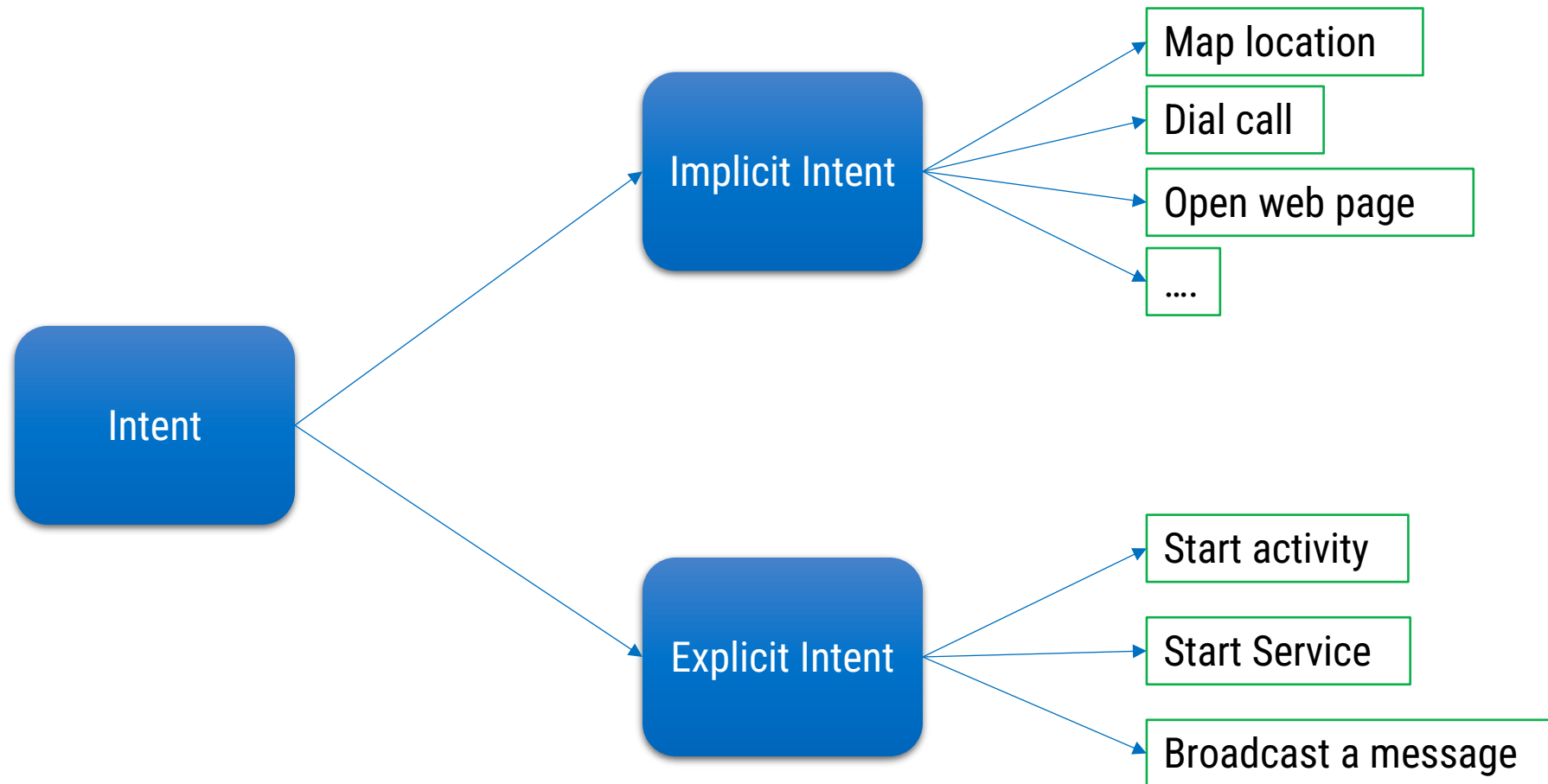
▶ Component

- Specifies an explicit name of a component class to use for the intent. Normally this is determined by looking at the other information in the intent (the action, data/type, and categories) and matching that with a component that can handle it. If this attribute is set then none of the evaluation is performed, and this component is used exactly as is. By specifying this attribute, all of the other Intent attributes become optional.

▶ Extras

- This is a Bundle of any additional information. This can be used to provide extended information to the component. For example, if we have a action to send an e-mail message, we could also include extra pieces of data here to supply a subject, body, etc.

Intent Types



Explicit Intent

Start a new Activity (LoginActivity -> ProfileActivity)

```
Intent myIntent = new Intent(this, ProfileActivity.class);  
this.startActivity(myIntent);
```

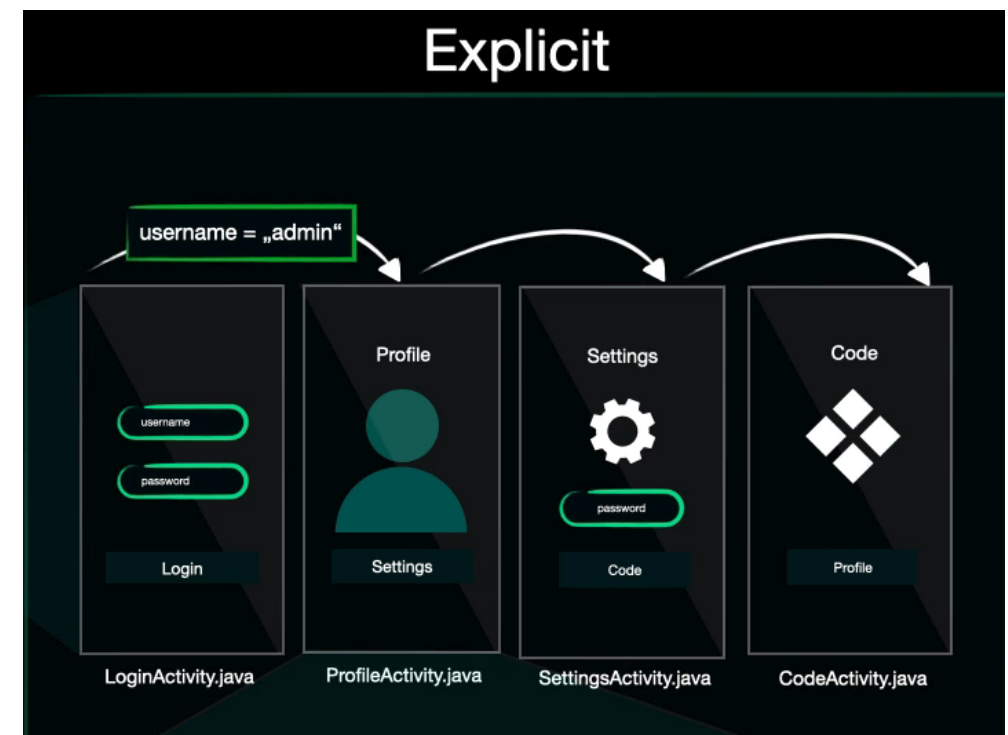
Start a new Activity with parameters

```
Intent myIntent = new Intent(this, ProfileActivity.class);  
myIntent.putExtra("username", "admin")  
this.startActivity(myIntent);
```

@Override

```
protected void onCreate(Bundle savedInstanceState) {  
    Intent intent = getIntent();  
    String user = intent.getStringExtra("username");  
}
```

Explicit Intents have specified a component, which provides the exact class to be run

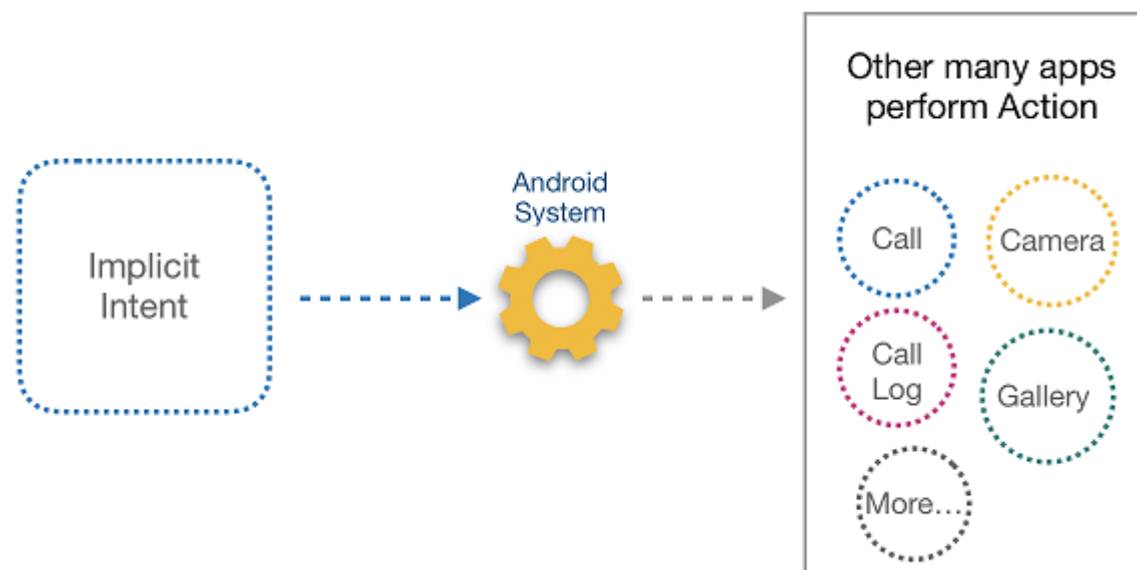


Implicit Intent

An implicit intent specifies an action that can invoke any app on the device able to perform the action

Send an implicit intent

```
Intent intent = new Intent(Intent.ACTION_VIEW);  
intent.setData(Uri.parse(uriString: "https://www.google.com/"));  
startActivity(intent);
```



Sending Intent

Start an activity

```
Intent intent = new Intent( packageContext: this, ExampleActivity.class);  
startActivity(intent);
```

Start a service

```
Intent intent = new Intent( packageContext: this, ExampleService.class);  
startService(intent);
```

Send broadcast

```
Intent intent = new Intent(Intent.ACTION_VIEW);  
intent.setData(Uri.parse( uriString: "https://www.google.com/"));  
sendBroadcast(intent);
```

Receive Intent

Get intent in activity

```
@Override
protected void onStart() {
    super.onStart();
    Intent intent = getIntent();
}
```

Get intent in service

```
@Override
public int onStartCommand(Intent intent, int flags, int startId) {
    return super.onStartCommand(intent, flags, startId);
}
```

Send broadcast

```
private BroadcastReceiver broadcastReceiver = new BroadcastReceiver() {
    @Override
    public void onReceive(Context context, Intent intent) {
        //filter intent by action
    }
};
```

▶ What are Intent Filters?

- Filters informs the system which implicit intents a component can handle
- If a component does not have any intent filters, it can receive only explicit intents.
- A component with filters can receive both explicit and implicit intents.
- A component has separate filters for each job it can do

Intent filter

- ▶ Inside the <intent-filter>, you can specify the type of intents to accept using one or more of these three elements:

<action>

Declares the intent action accepted, in the name attribute. The value must be the literal string value of an action, not the class constant.

<data>

Declares the type of data accepted, using one or more attributes that specify various aspects of the data URI (scheme, host, port, path) and MIME type.

<category>

Declares the intent category accepted, in the name attribute. The value must be the literal string value of an action, not the class constant.

```
<activity android:name=".MainActivity">
  <intent-filter>
    <action android:name="android.intent.action.SEND"/>
    <category android:name="android.intent.category.DEFAULT"/>
    <data android:mimeType="text/plain"/>
  </intent-filter>
</activity>
```


► How is Intent filter created?

- Create in AndroidManifest.xml

```
<activity android:name=".MainActivity">
    <intent-filter>
        <action android:name="android.intent.action.SEND"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <data android:mimeType="text/plain"/>
    </intent-filter>
</activity>
```

- Create in java class. Use for Broadcast receiver

```
@Override
protected void onStart() {
    super.onStart();
    IntentFilter intentFilter = new IntentFilter();
    intentFilter.addAction(Intent.ACTION_VIEW);
    intentFilter.addCategory(Intent.CATEGORY_DEFAULT);
    registerReceiver(broadcastReceiver, intentFilter);
}
```

▶ PendingIntent

- a wrapper around an Intent object.
- The primary purpose of a PendingIntent is to grant permission to a foreign application to use the contained Intent as if it were executed from your app's own process.

▶ Create pending intent

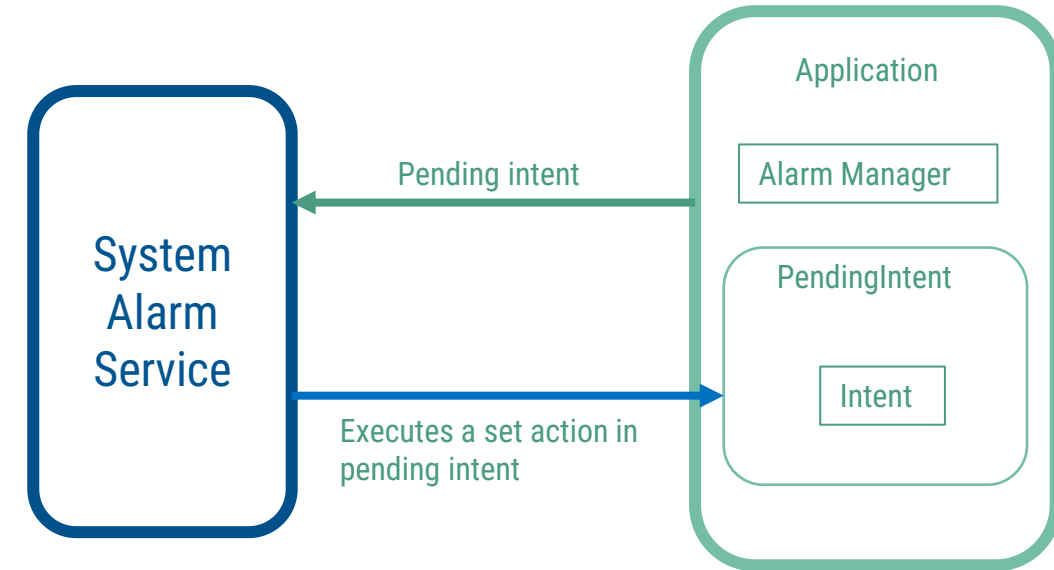
- `PendingIntent.getActivity()` for an Intent that starts an Activity.
- `PendingIntent.getService()` for an Intent that starts a Service.
- `PendingIntent.getBroadcast()` for an Intent that starts a BroadcastReceiver.

Pending Intent Example

Create Pending intent

```
private PendingIntent mPendingIntent;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    Intent alarmIntent = new Intent( packageContext: MainActivity.this,
                                   ExampleActivity.class);
    mPendingIntent = PendingIntent.getBroadcast( context: MainActivity.this,
                                                requestCode: 0, alarmIntent, PendingIntent.FLAG_UPDATE_CURRENT);
}
```



Create Alarm manager and schedule the alarm with pending intent

```
private void startAlarm() {
    AlarmManager alarmManager = (AlarmManager) getSystemService(Context.ALARM_SERVICE);
    alarmManager.setRepeating(AlarmManager.ELAPSED_REALTIME_WAKEUP,
                             triggerAtMillis: 0, AlarmManager.INTERVAL_FIFTEEN_MINUTES, mPendingIntent);
}
```

After 15 minutes, alarmIntent will be performed

Standard Activity Actions

- ACTION_MAIN: Start as a main entry point, does not expect to receive data.
- ACTION_VIEW: Display the data to the user.
- ACTION_ATTACH_DATA: Used to indicate that some piece of data should be attached to some other place.
- ACTION_EDIT: Provide explicit editable access to the given data.
- ACTION_PICK: Pick an item from the data, returning what was selected.
- ACTION_CHOOSER: Display an activity chooser, allowing the user to pick what they want to before proceeding
- ACTION_GET_CONTENT: Allow the user to select a particular kind of data and return it
- ACTION_DIAL: Dial a number as specified by the data.
- ACTION_CALL: Perform a call to someone specified by the data.
- ACTION_SEND: Deliver some data to someone else.

Standard Activity Actions

- ACTION_SENDTO: Send a message to someone specified by the data.
- ACTION_ANSWER: Handle an incoming phone call.
- ACTION_INSERT: Insert an empty item into the given container.
- ACTION_DELETE: Delete the given data from its container.
- ACTION_RUN: Run the data, whatever that means.
- ACTION_SYNC: Perform a data synchronization.
- ACTION_PICK_ACTIVITY: Pick an activity given an intent, returning the class selected.
- ACTION_SEARCH: Perform a search.
- ACTION_WEB_SEARCH: Perform a web search.
- ACTION_FACTORY_TEST: Main entry point for factory tests

Standard Broadcast Actions

- ACTION_TIME_TICK: The current time has changed. Sent every minute.
- ACTION_TIME_CHANGED: The time was set.
- ACTION_TIMEZONE_CHANGED: The timezone has changed.
- ACTION_BOOT_COMPLETED: This is broadcast once, after the user has finished booting.
- ACTION_PACKAGE_ADDED: A new application package has been installed on the device.
- ACTION_PACKAGE_CHANGED: An existing application package has been changed.
- ACTION_PACKAGE_REMOVED: An existing application package has been removed from the device.
- ACTION_PACKAGE_RESTARTED: The user has restarted a package, and all of its processes have been killed

Standard Broadcast Actions

- ACTION_PACKAGE_DATA_CLEARED: The user has cleared the data of a package.
- ACTION_PACKAGES_SUSPENDED: Packages have been suspended
- ACTION_PACKAGES_UNSUSPENDED: Packages have been unsuspended.
- ACTION_UID_REMOVED: A uid has been removed from the system.
- ACTION_BATTERY_CHANGED: This is a *sticky broadcast* containing the charging state, level, and other information about the battery.
- ACTION_POWER_CONNECTED: External power has been connected to the device.
- ACTION_POWER_DISCONNECTED: External power has been removed from the device.
- ACTION_SHUTDOWN: Device is shutting down.

Standard Category

- CATEGORY_DEFAULT: Set if the activity should be an option for the default action (center press) to perform on a piece of data.
- CATEGORY_BROWSABLE: Activities that can be safely invoked from a browser must support this category.
- CATEGORY_TAB: Intended to be used as a tab inside of a containing TabActivity.
- CATEGORY_ALTERNATIVE: Set if the activity should be considered as an alternative action to the data the user is currently viewing.
- CATEGORY_SELECTED_ALTERNATIVE: Set if the activity should be considered as an alternative selection action to the data the user has currently selected.
- CATEGORY_LAUNCHER: Should be displayed in the top-level launcher.
- CATEGORY_INFO: Provides information about the package it is in

Standard Category

- CATEGORY_HOME: This is the home activity, that is the first activity that is displayed when the device boots.
- CATEGORY_PREFERENCE: This activity is a preference panel.
- CATEGORY_TEST: To be used as a test (not part of the normal user experience).
- CATEGORY_CAR_DOCK: An activity to run when device is inserted into a car dock.
- CATEGORY_DESK_DOCK: An activity to run when device is inserted into a desk dock.
- CATEGORY_LE_DESK_DOCK: An activity to run when device is inserted into a analog (low end) dock.
- CATEGORY_HE_DESK_DOCK: An activity to run when device is inserted into a digital (high end) dock.
- CATEGORY_CAR_MODE: Used to indicate that the activity can be used in a car environment.
- CATEGORY_APP_MARKET: This activity allows the user to browse and download new applications.
- CATEGORY_VR_HOME: An activity to use for the launcher when the device is placed in a VR Headset viewer.

Standard Extra Data

- EXTRA_ALARM_COUNT: Used as an int extra field in [AlarmManager](#) pending intents to tell the application being invoked how many pending alarms are being delivered with the intent.
- EXTRA_BCC: A String[] holding e-mail addresses that should be blind carbon copied.
- EXTRA_CC: A String[] holding e-mail addresses that should be carbon copied.
- EXTRA_CHANGED_COMPONENT_NAME_LIST: This field is part of [ACTION_PACKAGE_CHANGED](#), and contains a string array of all of the components that have changed.
- EXTRA_DATA_REMOVED: Used as a boolean extra field in [ACTION_PACKAGE_REMOVED](#) intents to indicate whether this represents a full uninstall (removing both the code and its data) or a partial uninstall (leaving its data, implying that this is an update).
- EXTRA_DOCK_STATE: Used as an int extra field in [ACTION_DOCK_EVENT](#) intents to request the dock state.
- EXTRA_DOCK_STATE_CAR: Used as an int value for [EXTRA_DOCK_STATE](#) to represent that the phone is in a car dock.
- EXTRA_DOCK_STATE_DESK: Used as an int value for [EXTRA_DOCK_STATE](#) to represent that the phone is in a desk dock.

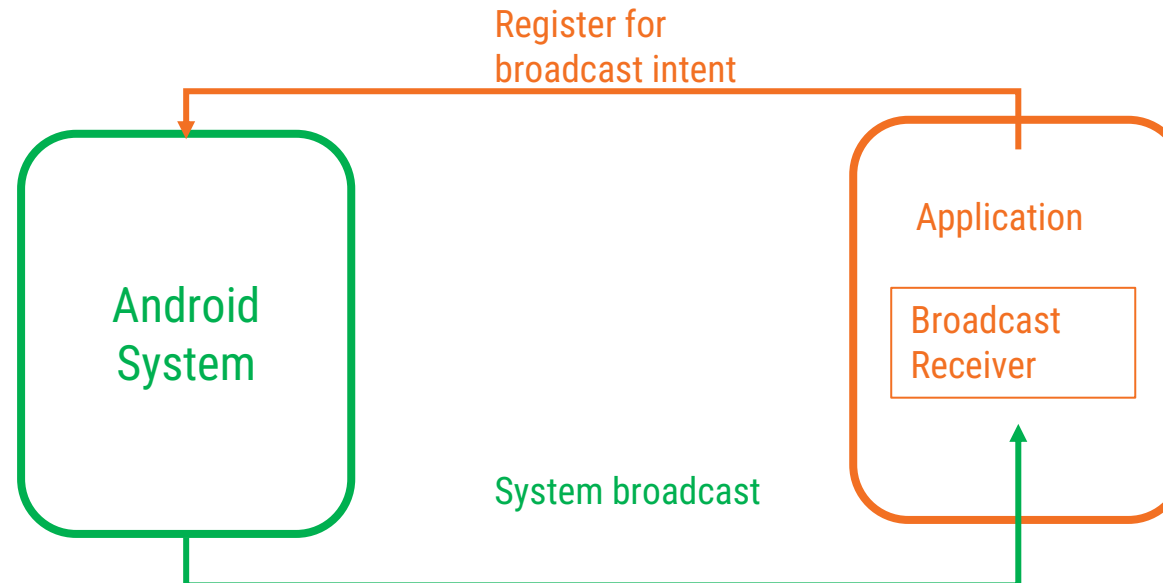
Standard Extra Data

- EXTRA_DOCK_STATE_UNDOCKED: Used as an int value for [EXTRA_DOCK_STATE](#) to represent that the phone is not in any dock.
- EXTRA_DONT_KILL_APP: Used as a boolean extra field in [ACTION_PACKAGE_REMOVED](#) or [ACTION_PACKAGE_CHANGED](#) intents to override the default action of restarting the application.
- EXTRA_EMAIL: A String[] holding e-mail addresses that should be delivered to.
- EXTRA_INITIAL_INTENTS: A Parcelable[] of [Intent](#) or [LabeledIntent](#) objects as set with [putExtra\(java.lang.String, android.os.Parcelable\[\]\)](#) to place at the front of the list of choices, when shown to the user with an [ACTION_CHOOSER](#)
- EXTRA_INTENT: An Intent describing the choices you would like shown with [ACTION_PICK_ACTIVITY](#) or [ACTION_CHOOSER](#).
- EXTRA_KEY_EVENT: A [KeyEvent](#) object containing the event that triggered the creation of the Intent it is in.
- EXTRA_ORIGINATING_URI: Used as a URI extra field with [ACTION_INSTALL_PACKAGE](#) and [ACTION_VIEW](#) to indicate the URI from which the local APK in the Intent data field originated from.
- EXTRA_PHONE_NUMBER: A String holding the phone number originally entered in [ACTION_NEW_OUTGOING_CALL](#), or the actual number to call in a [ACTION_CALL](#).

Broadcast Receiver

Broadcast Receiver

- Broadcast Receiver is one of the 4 major components in Android, with the purpose of listening to events, the state of the system emitted through the Intent so that programmers can handle system events on the side. in your application.
- Broadcast Receiver can work even when the application is shut down, that is, in the main background so it is often used with services.
- Two way to define Broadcast Receiver: AndroidManifest.xml and Java class



Manifest-Declared Receivers

- Create class Receiver extends BroadcastReceiver and implement method onReceive(Context, Intent)

```
public class Receiver extends BroadcastReceiver {  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        // TODO  
    }  
}
```

- Specify the <receiver> element in your app's manifest:

```
<receiver  
    android:name=".Receiver"  
    android:exported="true">  
    <intent-filter>  
        <action android:name="android.intent.action.AIRPLANE_MODE" />  
    </intent-filter>  
</receiver>
```

The intent filters specify the broadcast actions your receiver subscribes to.

Context-Registered Receivers

- Create class Receiver extends BroadcastReceiver and implement method onReceive(Context, Intent)

```
public class Receiver extends BroadcastReceiver {  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        // TODO  
    }  
}
```

- Create a IntentFilter to filter the action which you want to receive
- Call method registerReceiver(BroadcastReceiver, IntentFilter)

```
@Override  
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_main);  
    Receiver receiver = new Receiver();  
    IntentFilter intentFilter = new IntentFilter();  
    intentFilter.addAction(Intent.ACTION_AIRPLANE_MODE_CHANGED);  
    registerReceiver(receiver, intentFilter);  
}
```

Sending Broadcast

- Android provides three ways for apps to send broadcast:
 - ✓ The `sendOrderedBroadcast(Intent, String)` method sends broadcasts to one receiver at a time. As each receiver executes in turn, it can propagate a result to the next receiver, or it can completely abort the broadcast so that it won't be passed to other receivers. The order receivers run in can be controlled with the `android:priority` attribute of the matching intent-filter; receivers with the same priority will be run in an arbitrary order.
 - ✓ The `sendBroadcast(Intent)` method sends broadcasts to all receivers in an undefined order. This is called a Normal Broadcast. This is more efficient, but means that receivers cannot read results from other receivers, propagate data received from the broadcast, or abort the broadcast.

```
Intent intent = new Intent();
intent.setAction("com.example.broadcast.MY_NOTIFICATION");
intent.putExtra("data", "Nothing to see here, move along.");
sendBroadcast(intent);
```


Restricting broadcasts with permissions

Sending with permissions

- When you call `sendBroadcast(Intent, String)` or `sendOrderedBroadcast(Intent, String, BroadcastReceiver, Handler, int, String, Bundle)`, you can specify a permission parameter. Only receivers who have requested that permission with the tag in their manifest (and subsequently been granted the permission if it is dangerous) can receive the broadcast.

```
sendBroadcast(new Intent(BluetoothDevice.ACTION_FOUND),  
              Manifest.permission.BLUETOOTH_CONNECT)
```

- To receive the broadcast, the receiving app must request the permission as shown below:

```
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT"/>
```

Restricting broadcasts with permissions

Receiving with permissions

- If you specify a permission parameter when registering a broadcast receiver (either with `registerReceiver(BroadcastReceiver, IntentFilter, String, Handler)` or in `<receiver>` tag in your manifest), then only broadcasters who have requested the permission with the `<uses-permission>` tag in their manifest (and subsequently been granted the permission if it is dangerous) can send an Intent to the receiver. Example:
- receiving app has a manifest-declared receiver as shown below:

```
<receiver android:name=".MyBroadcastReceiver"
    android:permission="android.permission.BLUETOOTH_CONNECT">
    <intent-filter>
        <action android:name="android.intent.action.ACTION_FOUND" />
    </intent-filter>
</receiver>
```

- Or your receiving app has a context-registered receiver as shown below:

```
IntentFilter filter = new IntentFilter(Intent.ACTION_FOUND);
registerReceiver(receiver, filter, Manifest.permission.BLUETOOTH_CONNECT, null );
```

- Then, to be able to send broadcasts to those receivers, the sending app must request the permission as shown below:

```
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
```

THANK YOU!

